United States International University-Africa

EDUCATION TO TAKE YOU PLACES

Course Code& Title : Collaborative Software Development

Assessment : Practical Local VCS /Distributed VCS (individual)

Student Name : kyeong-je park

Student ID Number : 661211

Credit Card Fraud Detection Using The Random Forest Classification Model And GitHub Version Control

Table of contents

1.0 Project Overview **2**
introduction 4
2.0 PROJECT GOAL/OBJECTIVE **4**
General Objective: 4
3.0 METHODOLOGY 4
3.1 Version Control System (VCS) 4
3.2 Cloning from git **5**
3.3 Committing and pushing 6
Creating and merging branches 7
3.2 Algorithm / ML Model Overview **7**
Model Implementation Steps: 7
Download dataset **8**
Dataset management **8**
Importing libraries 8
Importing the dataset(csv file) 9
Data cleaning 10
Data visualization 13
Customer segmentation 30
Data modelling 40
Classification model **44**
Random forest classification 47
Random forest regression 48
Errors **49**

1.0 Project Overview

This project focuses on solving the problem of fraud detection in clothing , sports , and electronic supplies using machine learning techniques. The dataset used in this project is maintained transparently with the Creative Commons 4.0 license by Fabian Constante, Fernando Silva, and António Pereira through the Mendeley data repository. The dataset consists of roughly 180k transactions from supply chains used by the company DataCo Global for 3 years. The dataset can be downloaded from:
https://data.mendeley.com/datasets/8gx2fvg2k6/5
The dataset also contains features such as allowing in the use of Machine Learning Algorithms and R Software.

The machine learning model selected for this project is the Random Forest Classification model due to its high prediction accuracy. The model was trained and evaluated using performance metrics such as accuracy, precision, recall, and F1-score.
The overall performance of the model was overall very good, achieving an accuracy of 97% or above for all predictions. From an application perspective, the model can be used in fraud detection, customer segmentation, medical diagnosis, and stock price forecasting.

In addition, a Version Control System (VCS) was used to manage the project. Operations such as pulling, pushing, committing, branching, code review, and debugging were performed through google colab notebook and github. This helped in tracking changes, maintaining code integrity, and improving development efficiency.

1.1 Note

All code and instruction in this project have been slightly edited with help from google gemini and have been sourced from this practical assignment link: https://github.com/Peter26JumaOchieng/Version-Control-System-Practical-Assignment

2.0 Project Goal/Objective

2.1 General Objective:

To develop a machine learning model for fraud detection in clothing , sports , and electronic supplies while applying Version Control System (VCS) practices in project development.

Specific Objectives:

· To collect and preprocess data for analysis.

· To implement the random forrest classification model

· To evaluate the model using performance metrics.

· To apply VCS techniques such as version tracking, branching, and collaboration.

3.0 Methodology

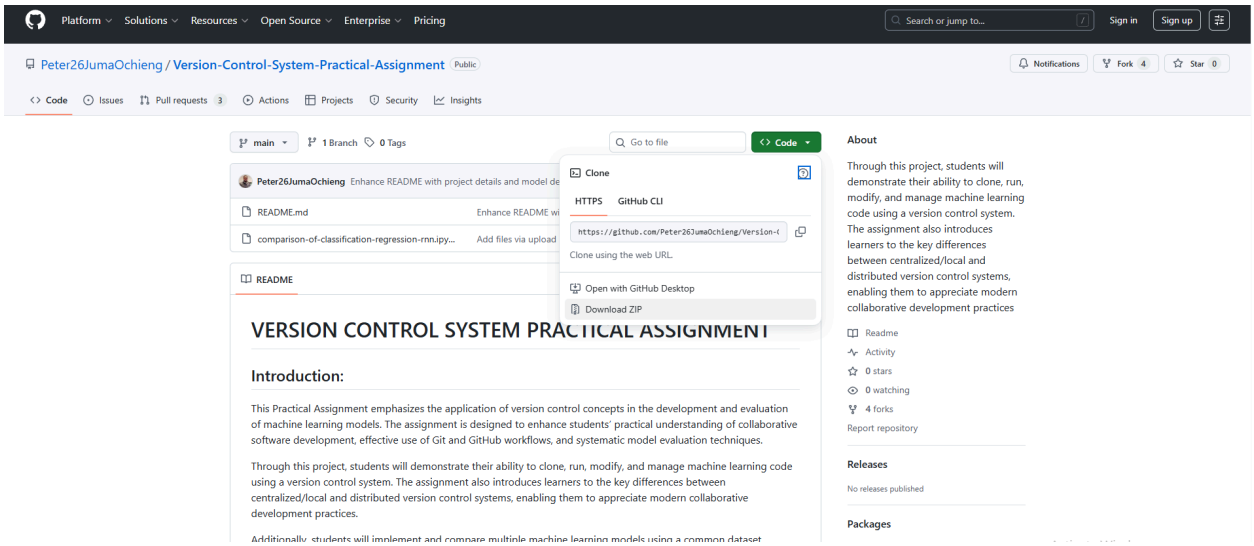
3.1 Version Control System (VCS)

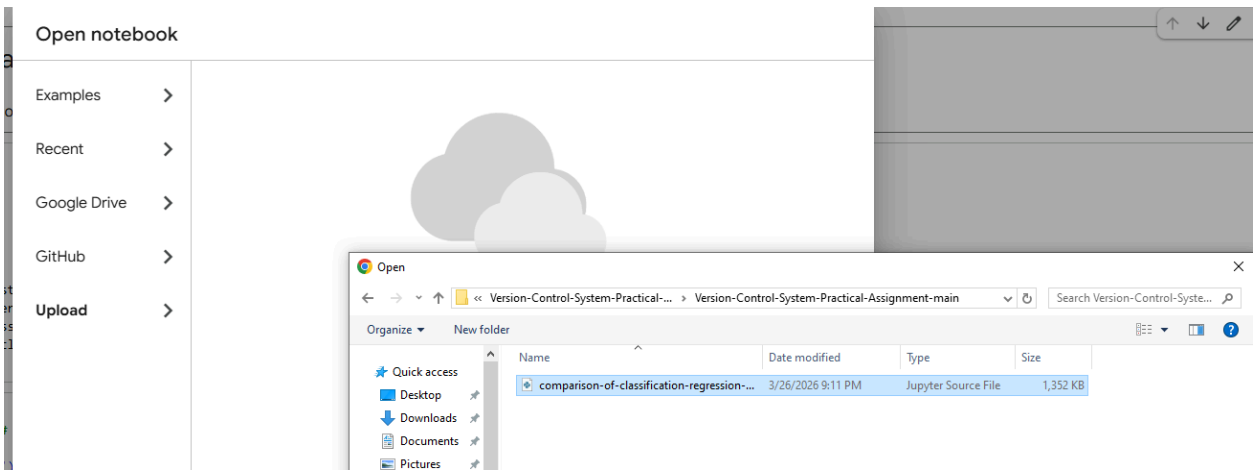
GitHub was used as the Version Control System for managing the project. The following steps were followed:

1. **Created a repository on GitHub named "app4080final"**
   (https://github.com/kjparkk/app4080final).
2. **Cloned the repository to the local machine by:**
   Downloading from github as a zip file directly and extracting then uploading directly to google colab
3. **Added project files to the repository.**
4. **Committed changes using:**
   Google colab github integration
5. **Pushed changes to GitHub using:**
   Google colab github integration
6. **Created branches for new features using:**
   Github user interface
7. **Merged branches after testing and debugging.**

3.2 Cloning from git

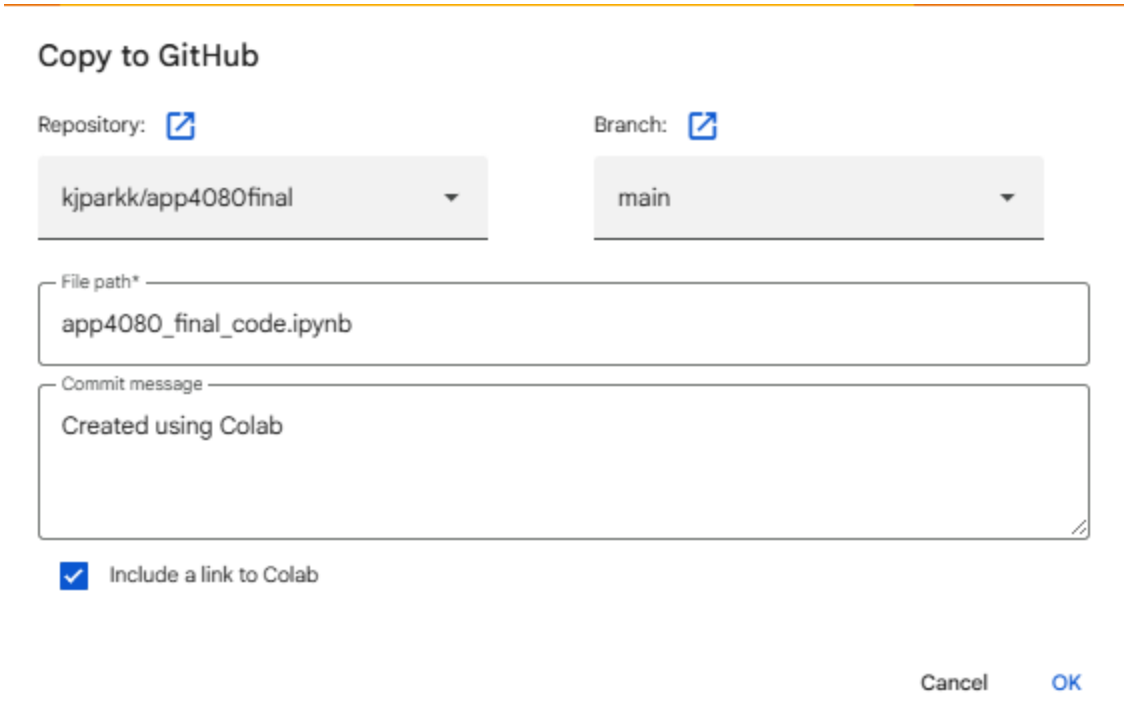
For this project, i have cloned from github by downloading the zip file of the project, unzipping it and uploading it to google colab, the steps are visualized below:





3.3 Committing and pushing

Committing and pushing to the repository can be done through linking google colab and the github repository using notebooks interface shown below:



3.4 Creating and merging branches

Cloning and merging branches can be done through github's user interface which i find is easier to navigate and visualise changes happening to the project

4.0 Algorithm / ML Model Overview

The **random forest classification model** works as follows:

Model Implementation Steps:

1. Data splitting into training and testing sets.
2. Model training using training data.
3. Model testing using unseen data.
4. Performance evaluation.

In this project:

- **LVCS (Local VCS)** was used for individual work.

· **DVCS (Distributed VCS)** (GitHub) supports collaboration and backup.

4.1 Download dataset

The dataset used in this project is maintained transparently with the Creative Commons 4.0 license by Fabian Constante, Fernando Silva, and António Pereira through the Mendeley data repository. The dataset consists of roughly 180k transactions from supply chains used by the company DataCo Global for 3 years. The dataset can be downloaded from:
https://data.mendeley.com/datasets/8gx2fvg2k6/5

4.2 Dataset management

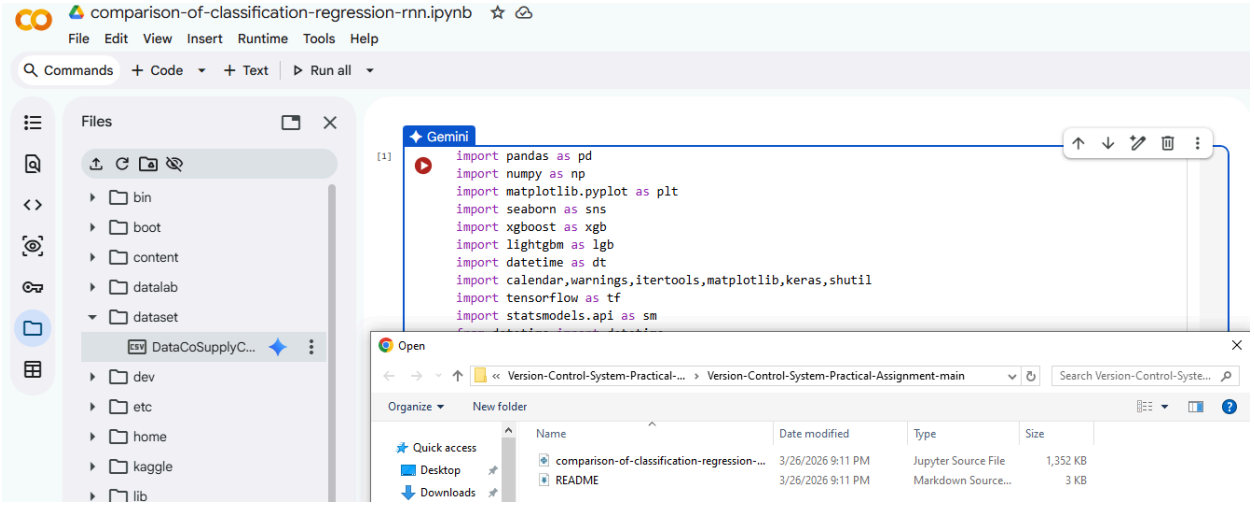
4.2.1 Importing libraries

The first block of code is dedicated to importing libraries required to run all the code within the project

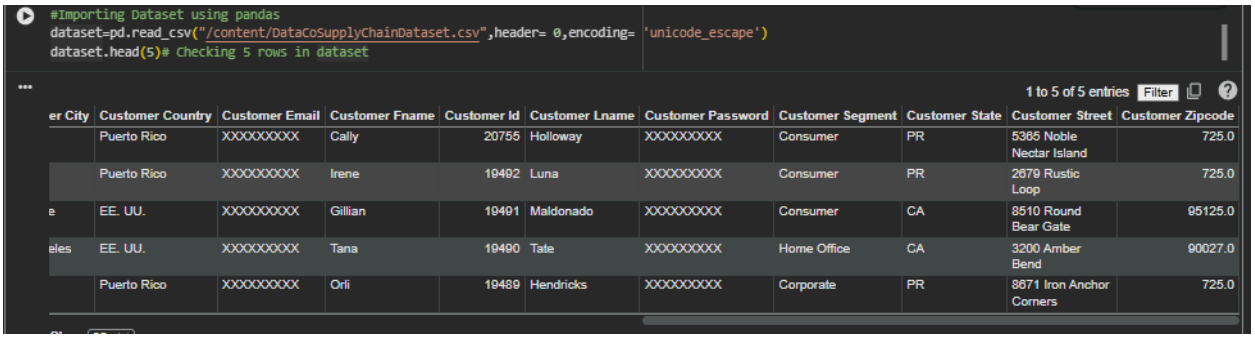
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import xgboost as xgb
import lightgbm as lgb
import datetime as dt
```

```
import calendar,warnings,itertools,matplotlib,keras,shutil
import tensorflow as tf
import statsmodels.api as sm
from datetime import datetime
from sklearn.model_selection import train_test_split,cross_val_score, cross_val_predict
from sklearn import svm,metrics,tree,preprocessing,linear_model
from sklearn.preprocessing import MinMaxScaler,StandardScaler
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeRegressor
from sklearn.linear_model import
Ridge,LinearRegression,LogisticRegression,ElasticNet, Lasso
from sklearn.ensemble import RandomForestRegressor,RandomForestClassifier,
GradientBoostingRegressor,BaggingClassifier,ExtraTre
from sklearn.metrics import
accuracy_score,mean_squared_error,recall_score,confusion_matrix,f1_score,roc_curve,
auc
from sklearn.datasets import load_iris,make_regression
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.kernel_ridge import KernelRidge
from keras import Sequential
from keras.layers import Dense
from IPython.core import display as ICD
#Hiding the warnings
warnings.filterwarnings('ignore')
```

4.2.2 Importing the dataset(csv file)



this code at segment 2 of the file shows which file the dataset is being read from, however, because the project is in colab so the dataset must be added again every time we want to start a new session of this project :

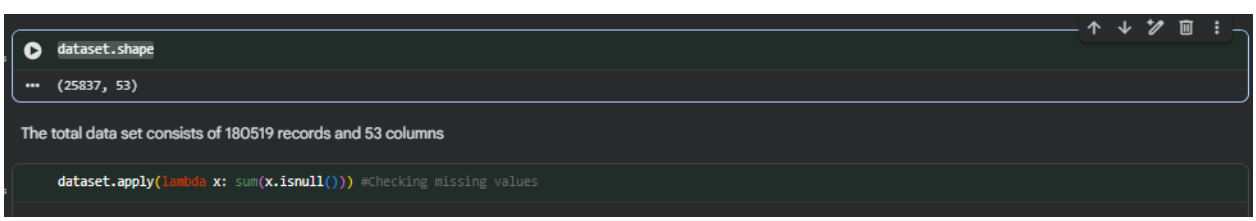


```
#Importing Dataset using pandas
dataset=pd.read_csv("/content/DataCoSupplyChainDataset.csv",header= 0,encoding= 'unicode_escape')
dataset.head(5)# Checking 5 rows in dataset
```

1 to 5 of 5 entries Filter

er City	Customer Country	Customer Email	Customer Fname	Customer Id	Customer Lname	Customer Password	Customer Segment	Customer State	Customer Street	Customer Zipcode
	Puerto Rico	XXXXXXXXX	Cally	20755	Holloway	XXXXXXXXX	Consumer	PR	5365 Noble Nectar Island	725.0
	Puerto Rico	XXXXXXXXX	Irene	19492	Luna	XXXXXXXXX	Consumer	PR	2679 Rustic Loop	725.0
e	EE. UU.	XXXXXXXXX	Gillian	19491	Maldonado	XXXXXXXXX	Consumer	CA	8510 Round Bear Gate	95125.0
eles	EE. UU.	XXXXXXXXX	Tana	19490	Tate	XXXXXXXXX	Home Office	CA	3200 Amber Bend	90027.0
	Puerto Rico	XXXXXXXXX	Orli	19489	Hendricks	XXXXXXXXX	Corporate	PR	8671 Iron Anchor Corners	725.0

4.2.3 Data cleaning



```
dataset.shape
```

```
(25837, 53)
```

The total data set consists of 180519 records and 53 columns

```
dataset.apply(lambda x: sum(x.isnull())) #Checking missing values
```

The full output for the second block of code is as follows:

```
Type   0
Days for shipping (real)      0
Days for shipment (scheduled)      0
Benefit per order      0
Sales per customer   0
Delivery Status        0
Late_delivery_risk   0
Category Id   0
Category Name        0
Customer City          0
Customer Country    0
Customer Email       0
Customer Fname      0
Customer Id  0
Customer Lname      0
Customer Password 0
Customer Segment   0
Customer State        0
Customer Street       0
Customer Zipcode    0
Department Id         0
Department Name    0
Latitude        0
Longitude      0
Market          0
Order City     0
```

Order Country 0

Order Customer Id 0

order date (DateOrders) 0

Order Id 0

Order Item Cardprod Id 0

Order Item Discount 0

Order Item Discount Rate 0

Order Item Id 0

Order Item Product Price 0

Order Item Profit Ratio 0

Order Item Quantity 0

Sales 0

Order Item Total 0

Order Profit Per Order 0

Order Region 0

Order State 0

Order Status 0

Order Zipcode 22407

Product Card Id 0

Product Category Id 0

Product Description 25837

Product Image 0

Product Name 1

Product Price 1

Product Status 1

shipping date (DateOrders) 1

Shipping Mode 1

```
# Adding first name and last name together to create new column
dataset['Customer Full Name'] = dataset['Customer Fname'].astype(str)+dataset['Customer Lname'].astype(str)
```

To make it easier for analysis some unimportant columns are dropped

```
data=dataset.drop(['Customer Email','Product Status','Customer Password','Customer Street','Customer Fname','Customer Lname',
          'Latitude','Longitude','Product Description','Product Image','Order Zipcode','shipping date (DateOrders)'],axis=1)
data.shape
```

```
(25837, 42)
```

There are 3 missing values in Customer Zipcode column. Since the missing values are just zip codes which are not very important these are replaced with zero before proceeding with data analysis.

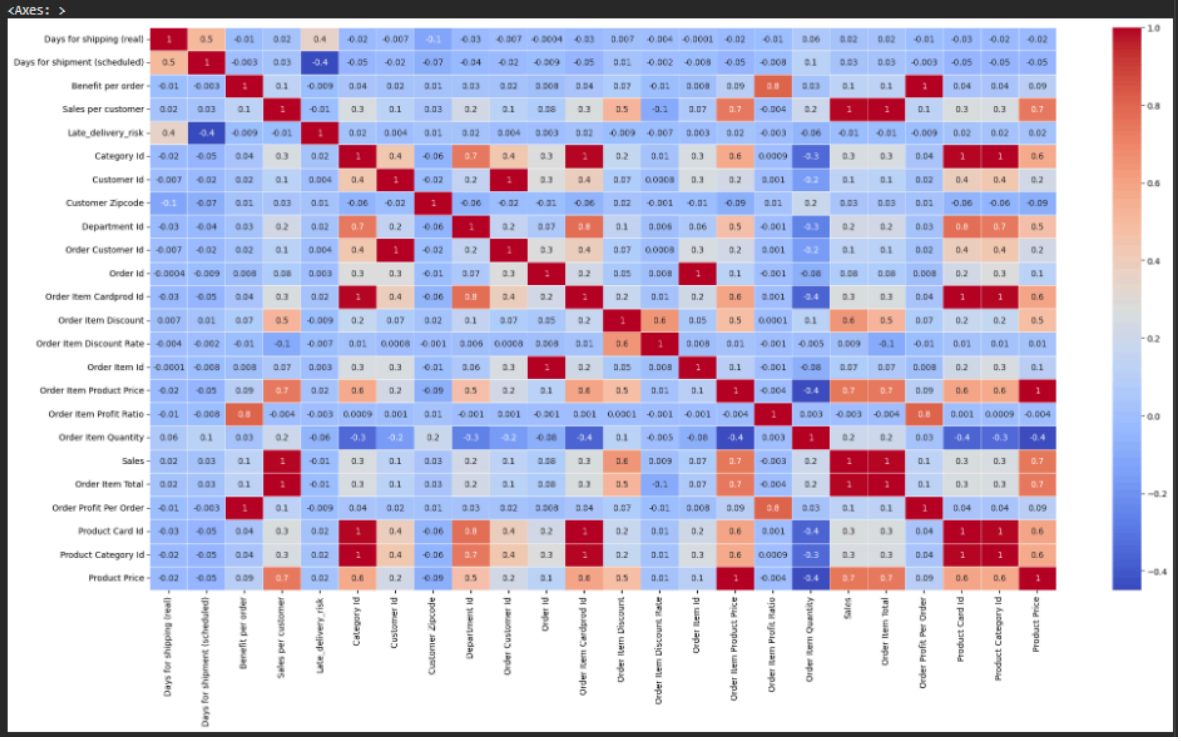
```
data['Customer Zipcode']=data['Customer Zipcode'].fillna(0)#Filling NaN columns with zero
```

4.2.4 Data visualization

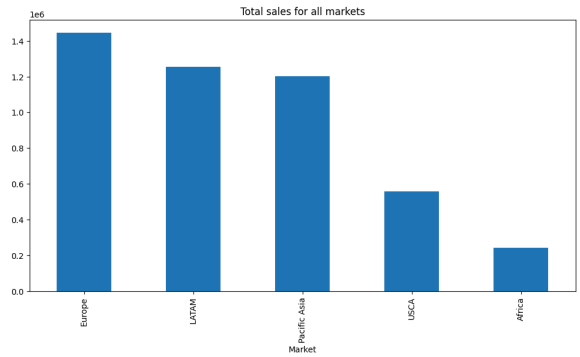
Here the data is put in a way that it can be visualized, all code relating to how the data is visualized along with the actual graphing are shown below as screenshots

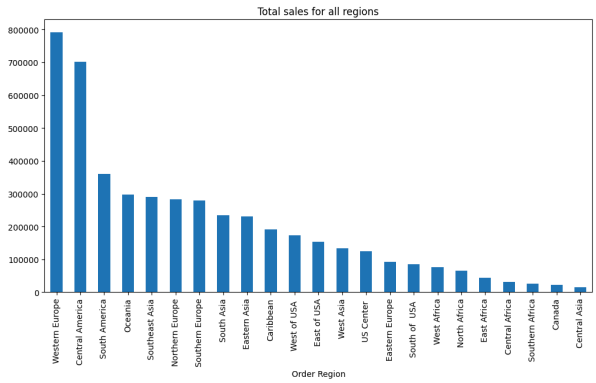
```
[10]
fig, ax = plt.subplots(figsize=(24,12))          # figsize
sns.heatmap(data.select_dtypes(include=np.number).corr(),annot=True,linewidths=.5,fmt='.1g',cmap= 'coolwarm') # Heatmap for correlation matrix
```

```
<Axes: >
```



```
market = data.groupby('Market') #Grouping by market
region = data.groupby('Order Region')
plt.figure(1)
market['Sales per customer'].sum().sort_values(ascending=False).plot.bar(figsize=(12,6), title="Total sales for all markets")
plt.figure(2)
region['Sales per customer'].sum().sort_values(ascending=False).plot.bar(figsize=(12,6), title="Total sales for all regions")
```



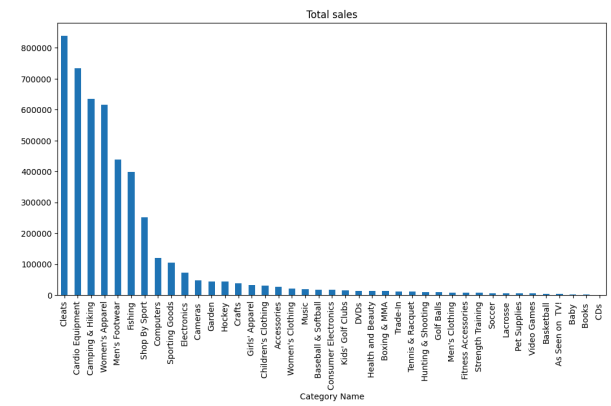


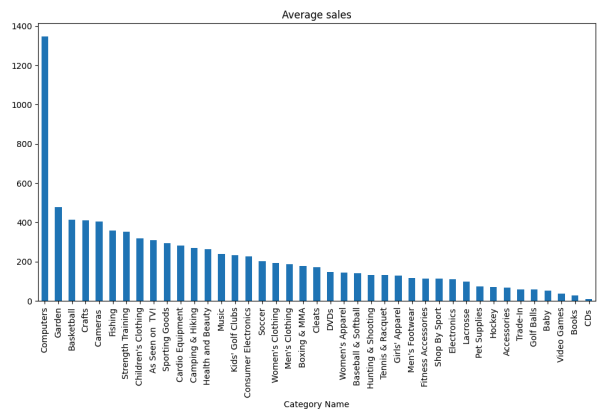
It could be seen from the graph that European market has the most number of sales whereas Africa has the least.In these markets western europe regions and central america recorded highest sales.

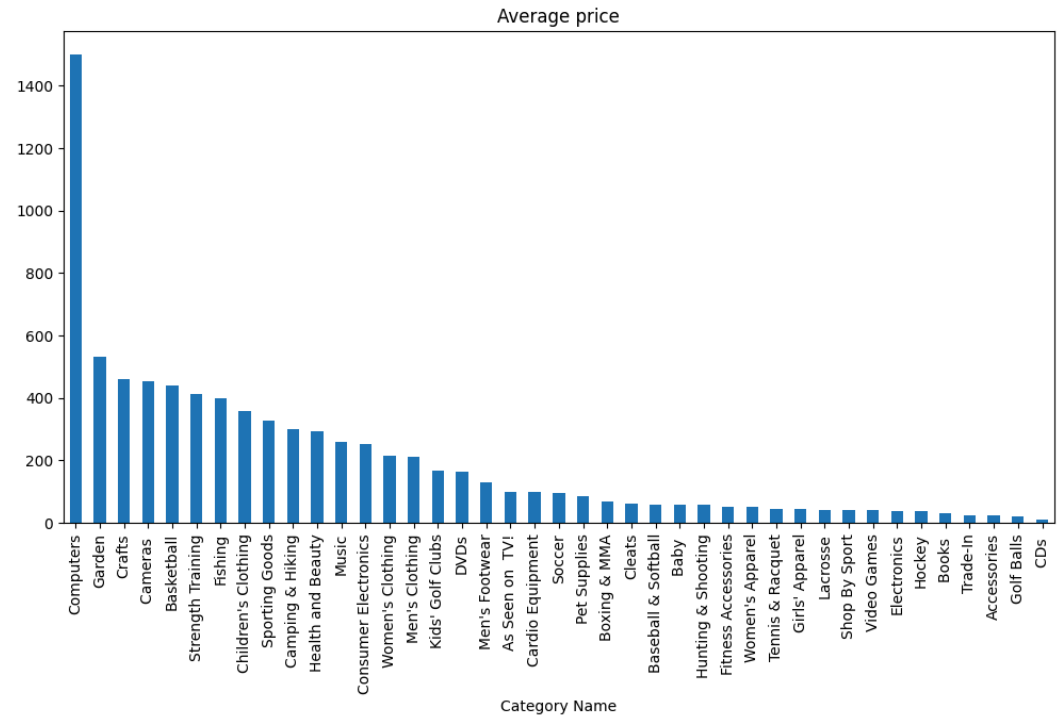
Which catergory of products has highest sales?The same method can be followed here to see the product category with highest sales

```
#Grouping all categories
```

```
cat = data.groupby('Category Name')
plt.figure(1)
# Total sum of sales for all categories
cat['Sales per customer'].sum().sort_values(ascending=False).plot.bar(figsize=(12,6),
title="Total sales")
# Mean sales for all categories
plt.figure(2)
cat['Sales per customer'].mean().sort_values(ascending=False).plot.bar(figsize=(12,6),
title="Average sales")
plt.figure(3)
# Mean prices for all categories
cat['Product Price'].mean().sort_values(ascending=False).plot.bar(figsize=(12,6),
title="Average price")
```

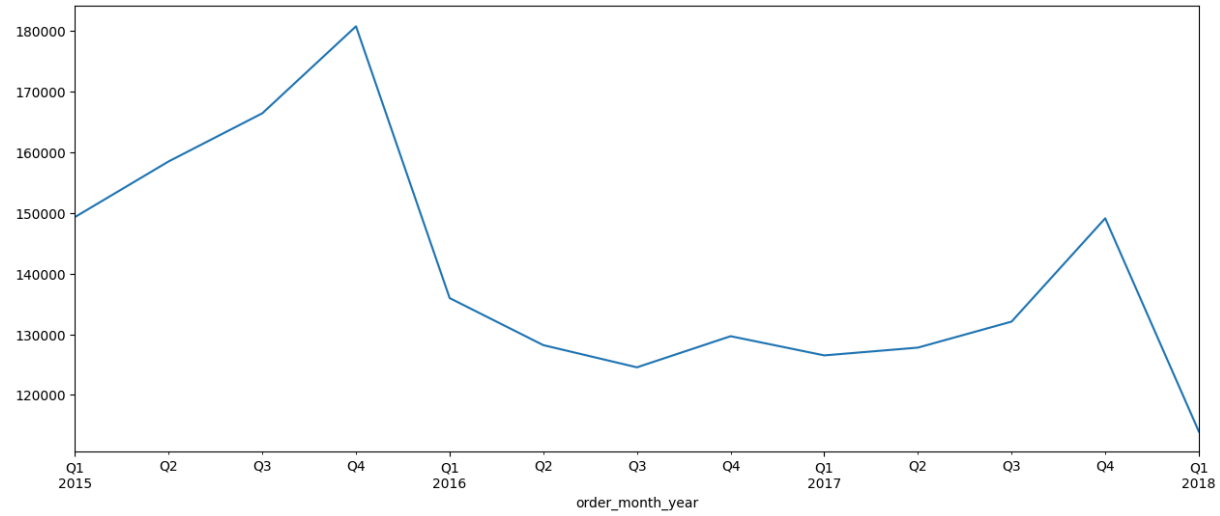






As we can see from fig 1 that the fishing category had most number of sales followed by the Cleats.However it is suprising to see that top 7 products with highest price on average are the most sold products on average with computers having almost 1350 sales despite price being 1500$. Since correlation was high between Price and Sales it will be intresting to see how price is impacting the sales for all the products to see the trend.

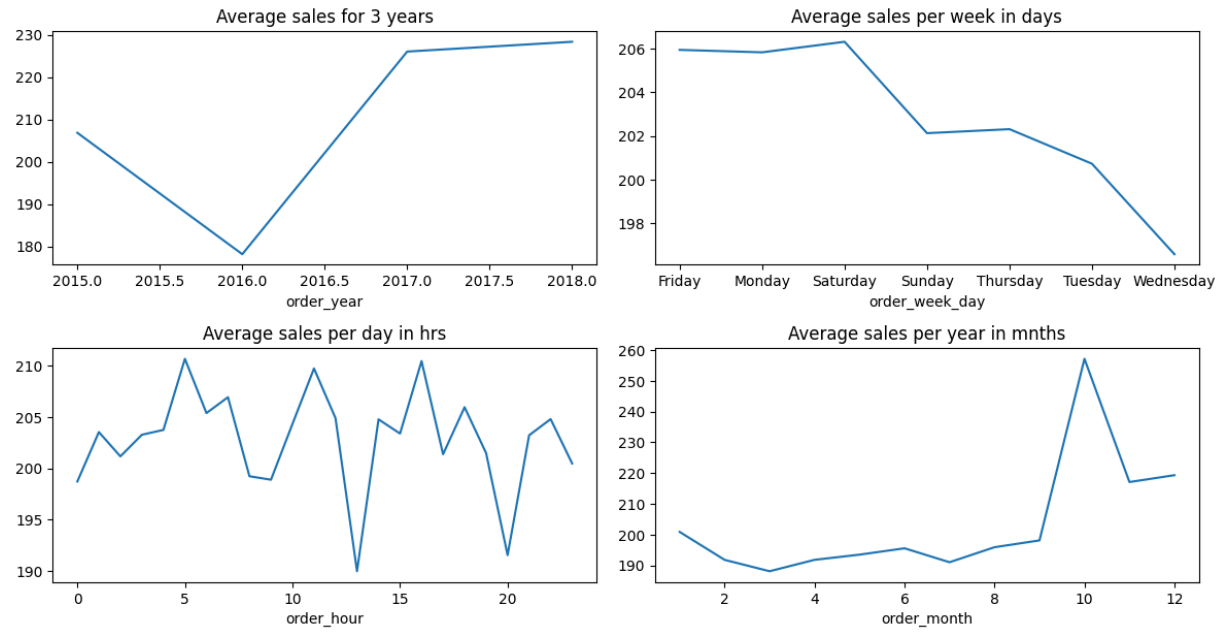
```
data.plot(x='Product Price', y='Sales per customer',linestyle='dotted',
    markerfacecolor='blue', markersize=12)
plt.title('Product Price vs Sales per customer')#title
plt.xlabel('Product Price')  # X-axis title
plt.ylabel('Sales per customer') # Y=axis title
plt.show()
```



By seeing above graph it can be seen that sales are consistent from Q1 2015 until Q3 of 2017 and suddenly dipped by Q1 2018. What is the purchase trend in week days,hours and months?

```
plt.figure(figsize=(10,12))
plt.subplot(4, 2, 1)
quater= data.groupby('order_year')
quater['Sales'].mean().plot(figsize=(12,12),title='Average sales for 3 years')
plt.subplot(4, 2, 2)
days=data.groupby("order_week_day")
days['Sales'].mean().plot(figsize=(12,12),title='Average sales per week in days')
plt.subplot(4, 2, 3)
hrs=data.groupby("order_hour")
hrs['Sales'].mean().plot(figsize=(12,12),title='Average sales per day in hrs')
plt.subplot(4, 2, 4)
mnth=data.groupby("order_month")
mnth['Sales'].mean().plot(figsize=(12,12),title='Average sales per year in mnths')
```

```
plt.tight_layout()
plt.show()
```

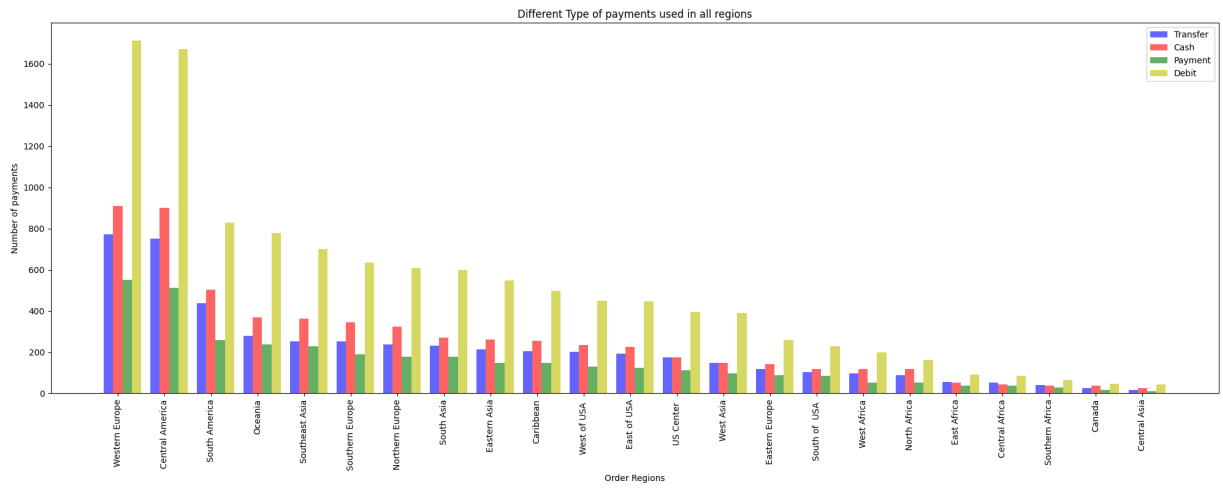


How price is impacting sales, when and which products are having more sales are found.The most number of orders came in October followed by November, and orders for all other months are consistent.Highest number of orders are placed by customers in 2017. Saturday recorded highest number of average sales and wednesday with the least number of sales. The average sales are consistent throughout the day irrespective of time with std of 3.

It is also important to know what type of payment method is being preferred by people to buy all these products in all regions? It can be found using .unique() method to see different payment methods.

```
data['Type'].unique()
```

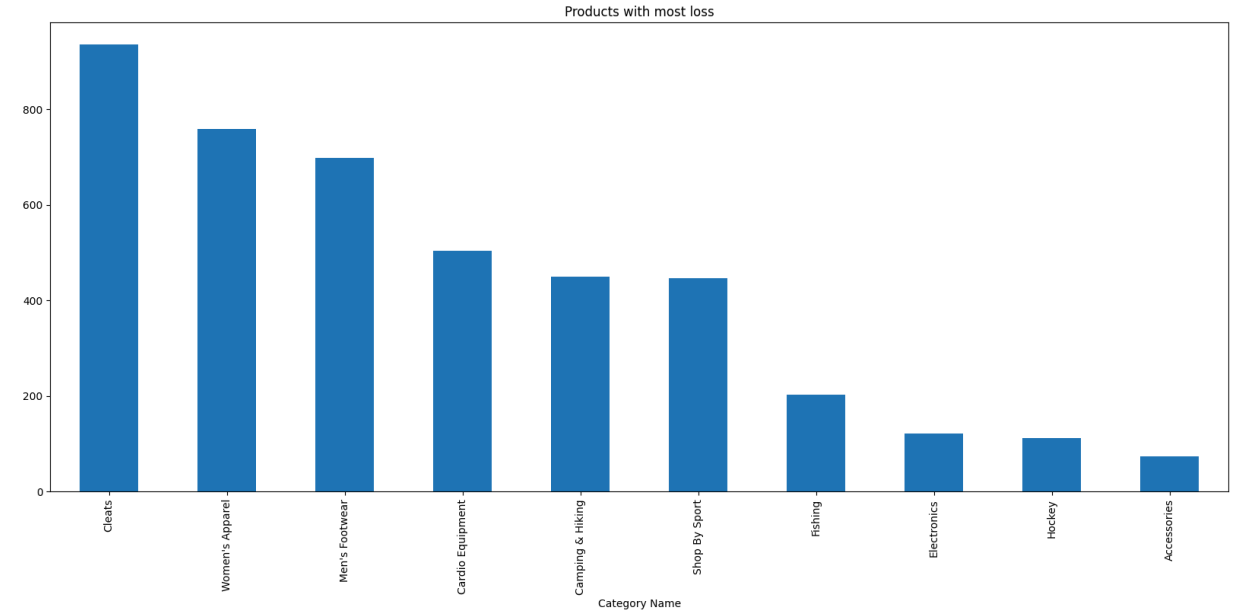
```
#xyz = data.groupby('Type')
xyz1 = data[(data['Type'] == 'TRANSFER')]
xyz2= data[(data['Type'] == 'CASH')]
xyz3= data[(data['Type'] == 'PAYMENT')]
xyz4= data[(data['Type'] == 'DEBIT')]
count1=xyz1['Order Region'].value_counts()
count2=xyz2['Order Region'].value_counts()
count3=xyz3['Order Region'].value_counts()
count4=xyz4['Order Region'].value_counts()
names=data['Order Region'].value_counts().keys()
n_groups=23
fig,ax = plt.subplots(figsize=(20,8))
index=np.arange(n_groups)
bar_width=0.2
opacity=0.6
type1=plt.bar(index,count1,bar_width,alpha=opacity,color='b',label='Transfer')
type2=plt.bar(index+bar_width,count2,bar_width,alpha=opacity,color='r',label='Cash')
type3=plt.bar(index+bar_width+bar_width,count3,bar_width,alpha=opacity,color='g',la
bel='Payment')
type4=plt.bar(index+bar_width+bar_width+bar_width,count4,bar_width,alpha=opacity,
color='y',label='Debit')
plt.xlabel('Order Regions')
plt.ylabel('Number of payments')
plt.title('Different Type of payments used in all regions')
plt.legend()
plt.xticks(index+bar_width,names,rotation=90)
plt.tight_layout()
plt.show()
```

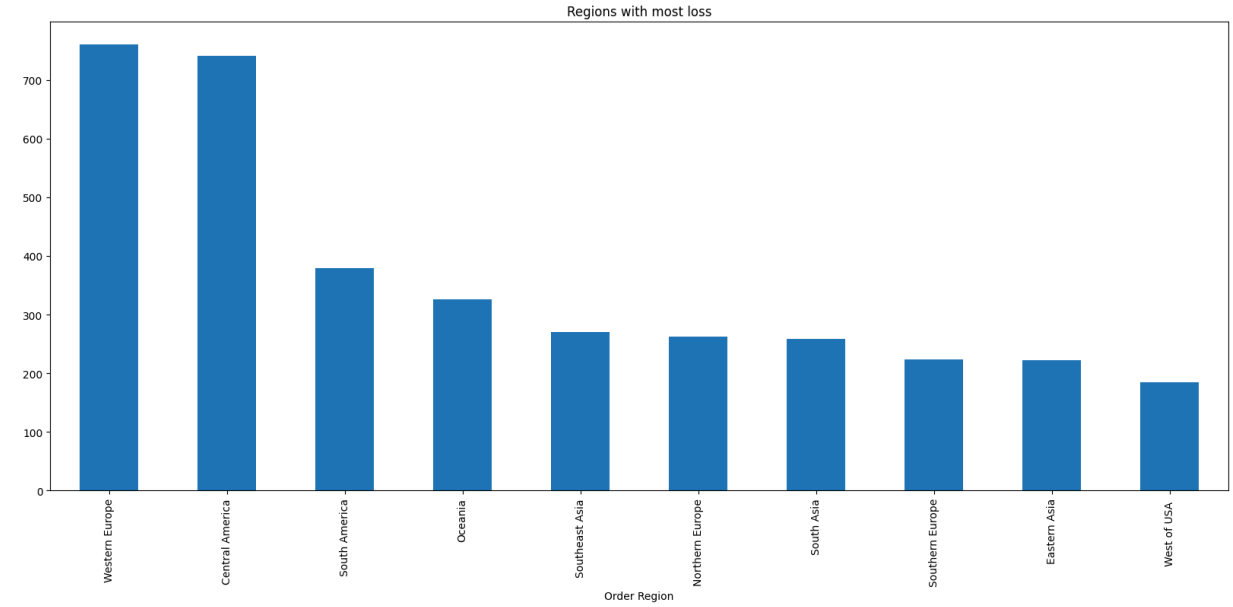


Debit type is most preferred payment method by people in all regions, Cash payment being the least preferred method.

Some products are having negative benefit per orders which indicates that the orders are generating loss of revenue to the company. Which products are these?

```
loss = data[(data['Benefit per order']<0)]
#Plotting top 10 products with most loss
plt.figure(1)
loss['Category Name'].value_counts().nlargest(10).plot.bar(figsize=(20,8),
title="Products with most loss")
plt.figure(2)
loss['Order Region'].value_counts().nlargest(10).plot.bar(figsize=(20,8), title="Regions
with most loss")
#Sum of total sales which are lost
print('Total revenue lost with orders',loss['Benefit per order'].sum())
```





The total loss sales are approximately 3.9 Millions which is an huge amount.It can be seen that Cleats is the category with most loss sales followed by Mens footwear.Most lost sales are happeing in central america and western europe region.This lost sales may have happened due to suspected frauds or late deliveries.

Finding which payment method is used to conduct frauds can be useful to prevent fraud from happening in future

```
#Checking type of payment used to conduct fraud other than Transfer
xyz = data[(data['Type'] != 'TRANSFER')&(data['Order Status'] ==
'SUSPECTED_FRAUD')]
xyz['Order Region'].value_counts()
```

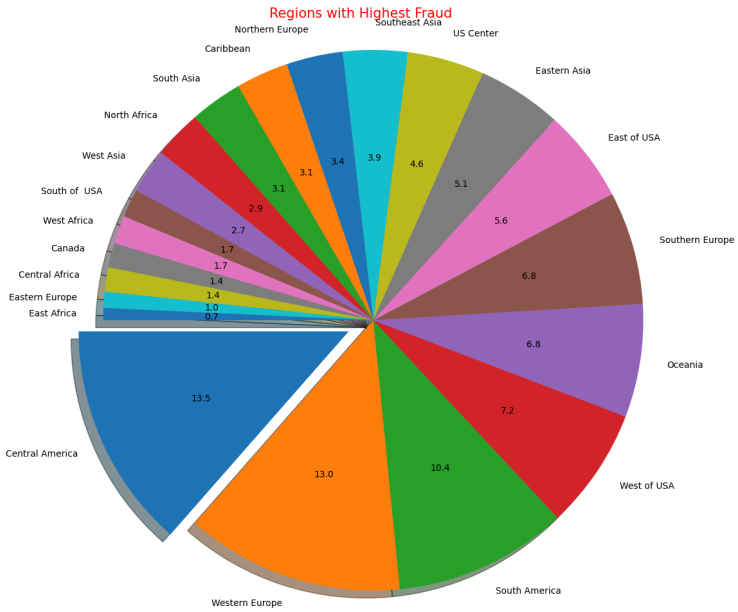
```
high_fraud = data[(data['Order Status'] == 'SUSPECTED_FRAUD') & (data['Type'] ==
'TRANSFER')]#separating orders with suspected fraud

# Get the value counts for 'Order Region'
fraud_regions_counts = high_fraud['Order Region'].value_counts()
num_regions = len(fraud_regions_counts)

# Create the explode tuple dynamically to match the number of regions
if num_regions > 0:
    explode_list = [0] * num_regions
    # Explode the first slice (which corresponds to the region with most frauds, as
value_counts() sorts by default)
    explode_list[0] = 0.1
    explode_tuple = tuple(explode_list)
else:
    explode_tuple = () # No regions to plot, so no explode needed

#Plotting pie chart with respect to order region
fraud=fraud_regions_counts.plot.pie(figsize=(24,12),
```

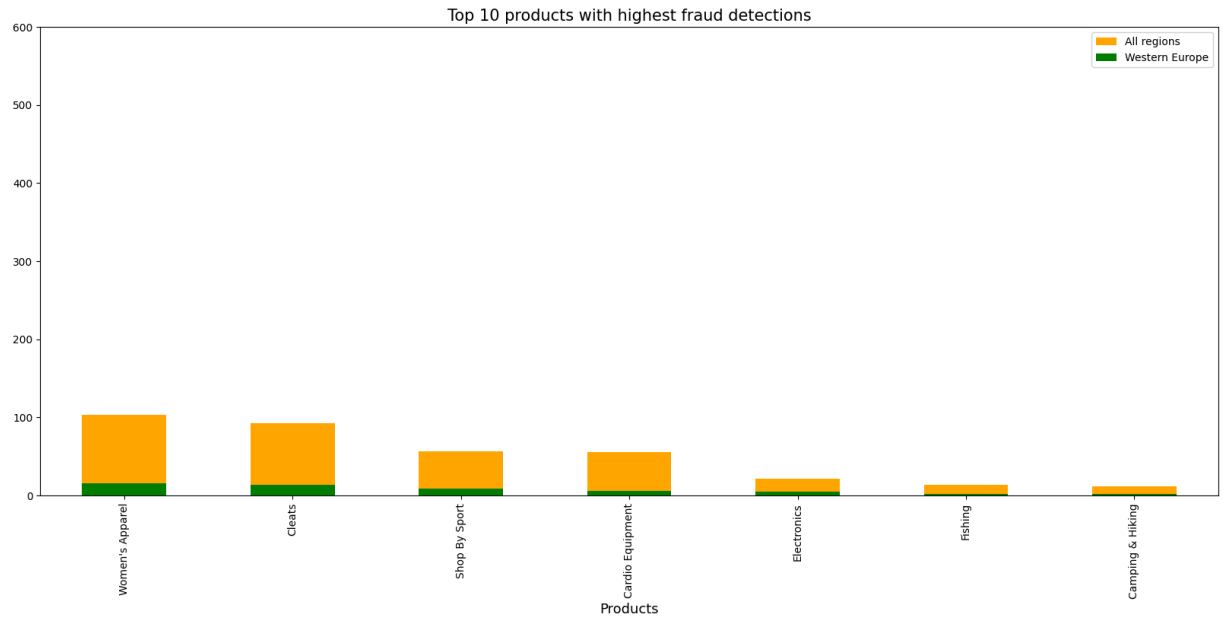
```
                                    startangle=180, explode=explode_tuple,
autopct='%.1f',shadow=True,)
plt.title("Regions with Highest Fraud",size=15,color='r') # Plotting title
plt.ylabel(" ")
fraud.axis('equal')
plt.show()
```



It can be observed that highest number of suspected fraud orders are from Western Europe which is approximately 17.4% of total orders followed by Central America with 15.5%. Which product is being suspected fraud the most?

```
high_fraud1 = data[(data['Order Status'] == 'SUSPECTED_FRAUD')] #
high_fraud2 = data[(data['Order Status'] == 'SUSPECTED_FRAUD') &(data['Order
Region'] == 'Western Europe')]
#Plotting bar chart for top 10 most suspected fraud department in all regions
fraud1=high_fraud1['Category
Name'].value_counts().nlargest(10).plot.bar(figsize=(20,8), title="Fraud
```

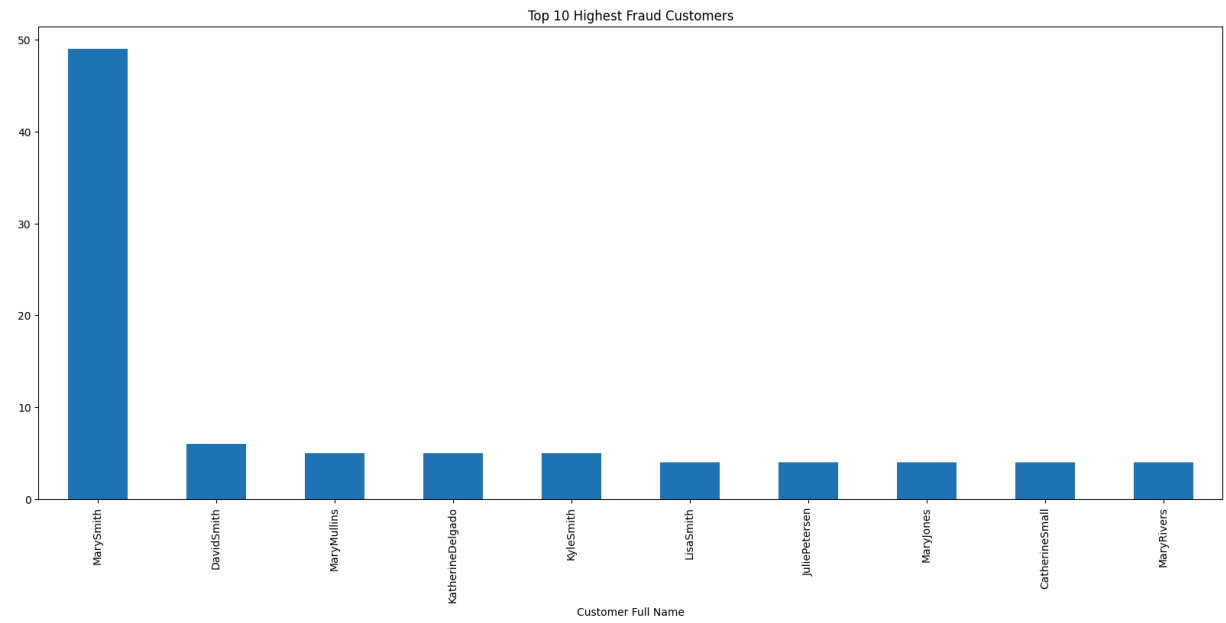
```
Category",color='orange')
#Plotting bar chart for top 10 most suspected fraud department in Western Europe
fraud2=high_fraud2['Category
Name'].value_counts().nlargest(10).plot.bar(figsize=(20,8), title="Fraud product in
Western Europe",color='green')
plt.legend(["All regions", "Western Europe"])
plt.title("Top 10 products with highest fraud detections", size=15)
plt.xlabel("Products", size=13)
plt.ylim(0,600)
plt.show()
```



It is very suprising to see that cleats department is being suspected to fraud the most followed by Men's footwear in all the regions and also in Western Europe.Which customers are conducting all these fraud?

```
#Filtering out suspected fruad orders
```

```
cus = data[(data['Order Status'] == 'SUSPECTED_FRAUD')]
#Top 10 customers with most fraud
cus['Customer Full Name'].value_counts().nlargest(10).plot.bar(figsize=(20,8),
title="Top 10 Highest Fraud Customers")
```



The customer named Mary Smith alone was responible for trying to conduct fraud 528 times which is very shocking .How much amount exactly did she conduct fraud orders?

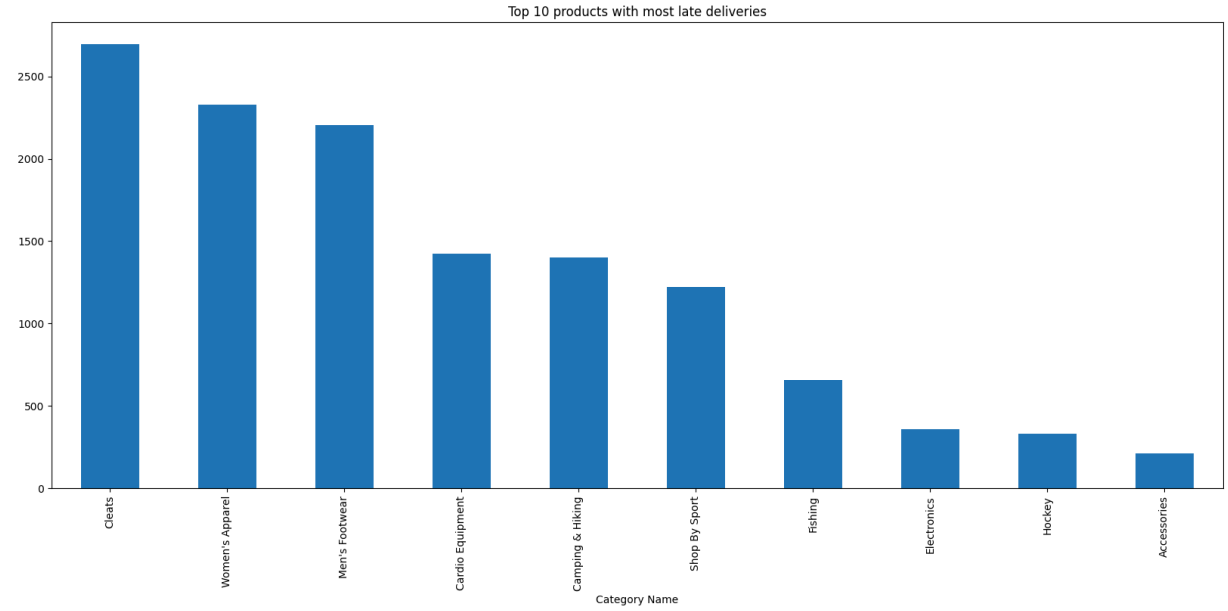
```
#Filtering orders of mary smith with suspected fraud
amount = data[(data['Customer Full Name'] == 'MarySmith')&(data['Order Status'] ==
'SUSPECTED_FRAUD')]
#Plotting bar chart for top 10 most suspected fraud customers
amount['Sales'].sum()
```

The total amount was almost 102k which is very huge amount.Since Mary was using different address every time when placing orders, a new customer id was issued each

time which makes it difficult to identify the customer and ban them. All these parameters should be taken into consideration to improve fraud detection algorithm so fraud can be identified more accurately.

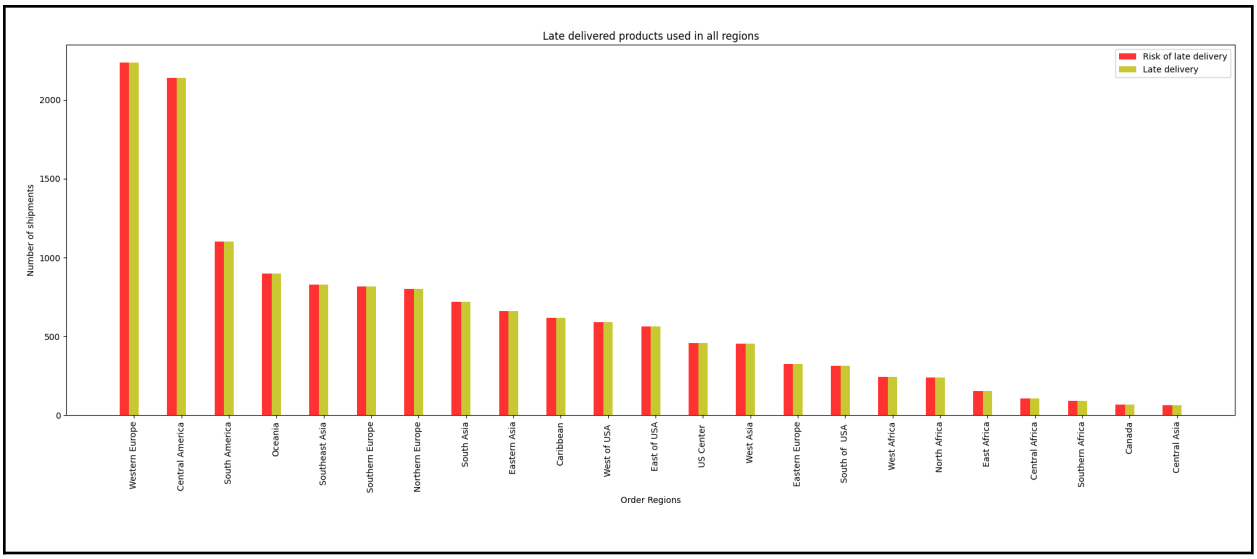
Delivering products to customer on time without late delivery is another important aspect for a supply chain company because customers will not be satisfied if products are not delivered on time. What category of products are being delivered late the most?

```
#Filtering columns with late delivery status
late_delivery = data[(data['Delivery Status'] == 'Late delivery')]
#Top 10 products with most late deliveries
late_delivery['Category Name'].value_counts().nlargest(10).plot.bar(figsize=(20,8),
title="Top 10 products with most late deliveries")
```



It can be seen that orders with Cleats department is getting delayed the most followed by Men's Footwear.For some orders risk of late delivery is given in data.The products with late delivery risk are compared with late delivered products.

```
#Filtering orders with late delivery risk
xyz1 = data[(data['Late_delivery_risk'] == 1)]
#Filtering late delivered orders
xyz2 = data[(data['Delivery Status'] == 'Late delivery')]
count1=xyz1['Order Region'].value_counts()
count2=xyz2['Order Region'].value_counts()
#Index names
names=data['Order Region'].value_counts().keys()
n_groups=23
fig,ax = plt.subplots(figsize=(20,8)) #Figure size
index=np.arange(n_groups)
bar_width=0.2
opacity=0.8
type1=plt.bar(index,count1,bar_width,alpha=opacity,color='r',label='Risk of late
delivery')
type2=plt.bar(index+bar_width,count2,bar_width,alpha=opacity,color='y',label='Late
delivery')
plt.xlabel('Order Regions')
plt.ylabel('Number of shipments')
plt.title('Late delivered products used in all regions')
plt.legend()
plt.xticks(index+bar_width,names,rotation=90)
plt.tight_layout()
plt.show()
```



Thus,it can be concluded that for all the products with late delivery risk irrespective of region the product is actually being delivered late,to avoid late delivery the company can ship products faster using better shipping method or schedule more time of Days for shipment so customers will know in advance when the products will reach them. It will be interesting to see the number of late deliveried orders for different types of shipment method in all regions.

```
#Filtering late delivery orders with standard class shipping
xyz1 = data[(data['Delivery Status'] == 'Late delivery') & (data['Shipping Mode'] ==
'Standard Class')]
#Filtering late delivery orders with first class shipping
xyz2 = data[(data['Delivery Status'] == 'Late delivery') & (data['Shipping Mode'] ==
'First Class')]
#Filtering late delivery orders with second class shipping
xyz3 = data[(data['Delivery Status'] == 'Late delivery') & (data['Shipping Mode'] ==
'Second Class')]
#Filtering late delivery orders with same day shipping
xyz4 = data[(data['Delivery Status'] == 'Late delivery') & (data['Shipping Mode'] ==
```

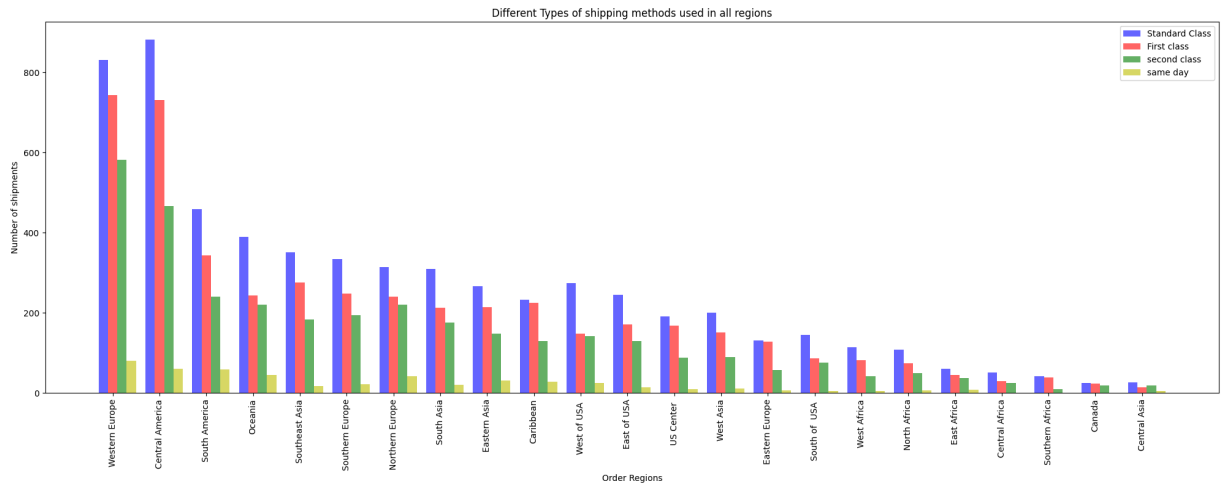
```
'Same Day')]
#Counting total values

# Get all unique order regions to ensure consistent indexing
all_regions = data['Order Region'].value_counts().index

count1 = xyz1['Order Region'].value_counts().reindex(all_regions, fill_value=0)
count2 = xyz2['Order Region'].value_counts().reindex(all_regions, fill_value=0)
count3 = xyz3['Order Region'].value_counts().reindex(all_regions, fill_value=0)
count4 = xyz4['Order Region'].value_counts().reindex(all_regions, fill_value=0)

#Index names
names=all_regions
n_groups=len(all_regions)
fig,ax = plt.subplots(figsize=(20,8))
index=np.arange(n_groups)
bar_width=0.2
opacity=0.6
type1=plt.bar(index,count1,bar_width,alpha=opacity,color='b',label='Standard Class')
type2=plt.bar(index+bar_width,count2,bar_width,alpha=opacity,color='r',label='First
class')
type3=plt.bar(index+bar_width+bar_width,count3,bar_width,alpha=opacity,color='g',la
bel='second class')
type4=plt.bar(index+bar_width+bar_width+bar_width,count4,bar_width,alpha=opacity,
color='y',label='same day')
plt.xlabel('Order Regions')
plt.ylabel('Number of shipments')
plt.title('Different Types of shipping methods used in all regions')
```

```
plt.legend()
plt.xticks(index+bar_width,names,rotation=90)
plt.tight_layout()
plt.show()
```



As expected the most number of late deliveries for all regions occured with standard class shipping,with same day shipping being the one with least number of late deliveries.Both the first class and second class shipping have almost equal number of late deliveries.

4.2.5 Customer segmentation

Understanding customer needs and targeting specific clusters of customers based on their need is one way for a supply chain company to increase number of customers and also to gain more profits.Since,purchase history of customers is already avaialble in the dataset, it can use RFM analysis for customer segmention. Even though there are so many different methods for customer segmentation,RFM analysis is being used because it utilizes numerical values to show Customer recency,frequency and monetary values and also the output results are easy to interpret.

```
#Calculating total price for which each order
data['TotalPrice'] = data['Order Item Quantity'] * data['Order Item Total']# Multiplying item price * Order quantity
```

```
data['order date (DateOrders)'].max() # Calculating when the last order come to check recency
```

The last order in the dataset was made on 2018-01-31. So the present time is set slightly above than the last order time for more accuracy of recency value.

```
#Present date was set to next day of the last order. i.e,2018-02-01
present = dt.datetime(2018,2,1)
data['order date (DateOrders)'] = pd.to_datetime(data['order date (DateOrders)'])
```

```
# Grouping all values into new data frame named customer segmentation
Customer_seg = data.groupby('Order Customer Id').agg({'order date (DateOrders)': lambda x: (present - x.max()).days, 'Order Id': lambda x: len(x), 'TotalPrice': lambda x: x.sum()})
#Changing order dates to int format
Customer_seg['order date (DateOrders)'] = Customer_seg['order date (DateOrders)'].astype(int)
# Renaming columns as R_Value,F_Value,M_Value
Customer_seg.rename(columns={'order date (DateOrders)': 'R_Value',
                    'Order Id': 'F_Value',
                    'TotalPrice': 'M_Value'}, inplace=True)
Customer_seg.head()
```

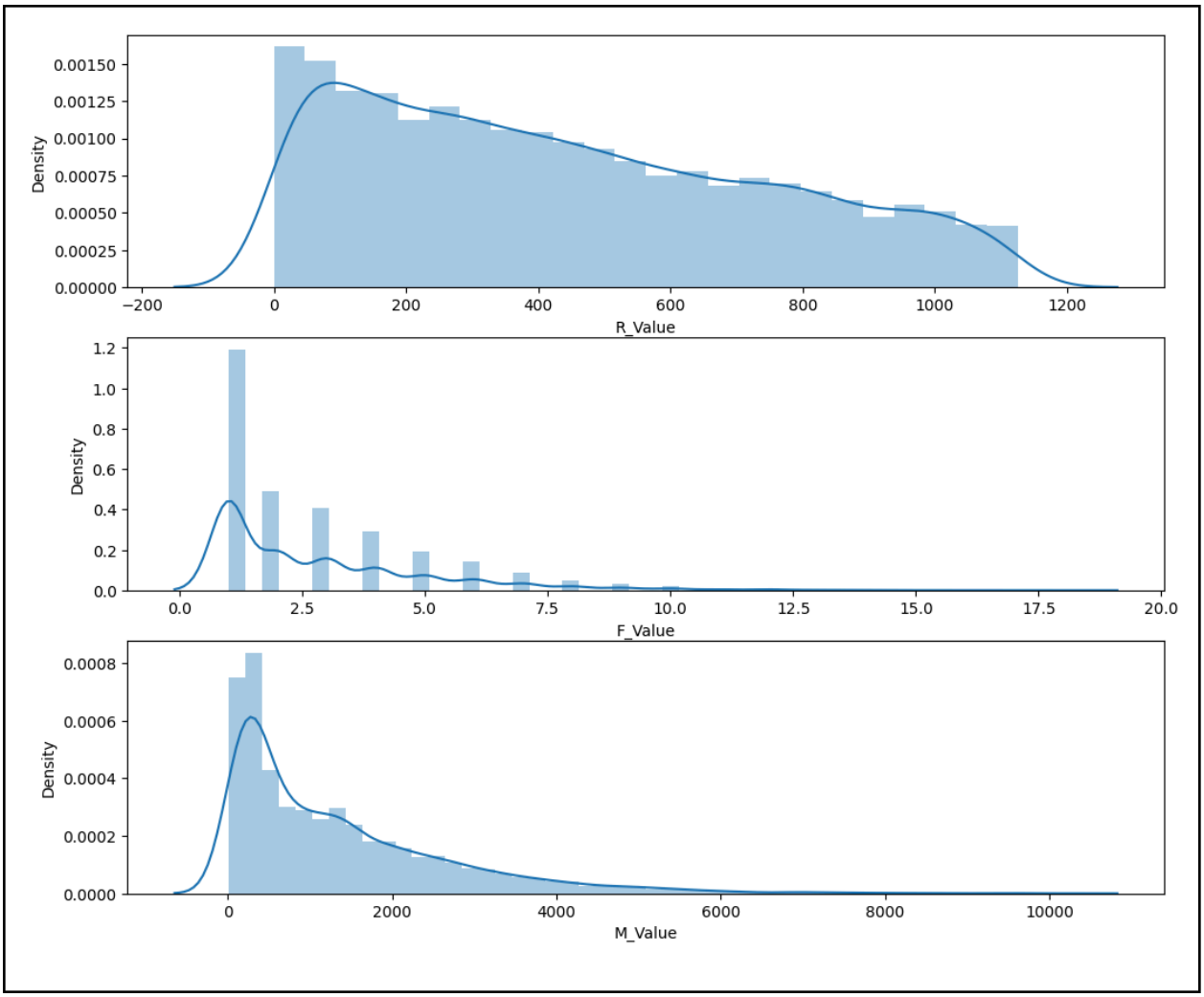
Order Customer Id	R_Value	F_Value	M_Value
1	792	1	2362.250061
2	280	1	839.800034
3	613	4	328.339998
4	380	2	1308.990032
5	457	4	1922.700043

R_Value(Recency) indicates how much time elapsed since a customer last order.

F_Value(Frequency) indicates how many times a customer ordered.

M_Value(Monetary value) tells us how much a customer has spent purchasing items.

```
plt.figure(figsize=(12,10)) # Figure size
plt.subplot(3, 1, 1)
sns.distplot(Customer_seg['R_Value'])# Plot distribution of R_Value
plt.subplot(3, 1, 2)
sns.distplot(Customer_seg['F_Value'])# Plot distribution of F_Value
plt.subplot(3, 1, 3)
sns.distplot(Customer_seg['M_Value'])# Plot distribution of M_Value
plt.show()
```



The total data is divided into 4 quantiles. The R_Value should be low because it indicates recent customer activity and F_value, M_Value should be high since they indicate frequency and total value of purchase. Function is defined to indicate quantiles as numerical values.

```
# R_Score should be minimum so 1st quantile is set as 1.
def R_Score(a,b,c):
    if a <= c[b][0.25]:
        return 1
    elif a <= c[b][0.50]:
```

```
        return 2
    elif a <= c[b][0.75]:
        return 3
    else:
        return 4
# The higher the F_Score,M_Score the better so 1st quantile is set as 4.
def FM_Score(x,y,z):
    if x <= z[y][0.25]:
        return 4
    elif x <= z[y][0.50]:
        return 3
    elif x <= z[y][0.75]:
        return 2
    else:
        return 1
```

```
# New column for R_Score to indicate numerical score between 1 to 4.
Customer_seg['R_Score'] = Customer_seg['R_Value'].apply(R_Score,
args=('R_Value',quantiles))
# New column for F_Score to indicate numerical score between 1 to 4.
Customer_seg['F_Score'] = Customer_seg['F_Value'].apply(FM_Score,
args=('F_Value',quantiles))
# New column for M_Score to indicate numerical score between 1 to 4.
Customer_seg['M_Score'] = Customer_seg['M_Value'].apply(FM_Score,
args=('M_Value',quantiles))
Customer_seg.head()
```

Order Customer Id	R_Value	F_Value	M_Value	R_Score	F_Score	M_Score
1	792	1	2362.250061	4	4	1
2	280	1	839.800034	2	4	3
3	613	4	328.339998	3	2	4
4	380	2	1308.990032	2	3	2
5	457	4	1922.700043	3	2	2

The individual scores of R,F,M are known.A column for combined RFM score is created.

```
#Adding R,F,M Scores to one new column
Customer_seg['RFM_Score'] = Customer_seg.R_Score.astype(str)+
Customer_seg.F_Score.astype(str) + Customer_seg.M_Score.astype(str)
Customer_seg.head()
```

Order Customer Id	R_Value	F_Value	M_Value	R_Score	F_Score	M_Score	RFM_Score
1	792	1	2362.250061	4	4	1	441
2	280	1	839.800034	2	4	3	243
3	613	4	328.339998	3	2	4	324
4	380	2	1308.990032	2	3	2	232
5	457	4	1922.700043	3	2	2	322

How many different customer segments are there in total can be found using unique() and len method.

```
count=Customer_seg['RFM_Score'].unique()
print(count)# Printing all Unique values
```

```
len(count)# Total count
```

```
['441' '243' '324' '232' '322' '421' '411' '433' '211' '223' '221' '332'
 '123' '244' '311' '422' '423' '344' '343' '112' '212' '312' '432' '111'
 '121' '442' '124' '222' '443' '323' '321' '233' '444' '234' '334' '412'
 '342' '133' '122' '144' '231' '434' '134' '413' '331' '333' '213' '132'
 '313' '431' '242' '424' '341' '142' '143' '141' '113' '241' '224' '131']
```

It can be seen that there are 33 different customer segments. To make it easier for segmentation individual R,F,M scores are added together

```
# Calculate RFM_Score
Customer_seg['RFM_Total_Score'] =
Customer_seg[['R_Score','F_Score','M_Score']].sum(axis=1)
Customer_seg['RFM_Total_Score'].unique()
```

```
array([ 9,  7,  6, 10,  4,  5,  8, 11,  3, 12])
```

There are 9 values in total for customer segmentation.Appropriate names were assigned for each value seperately.

```
# Define rfm_level function
def RFM_Total_Score(df):

    if (df['RFM_Total_Score'] >= 11):# For RFM score with values 11,12
        return 'Champions'
```

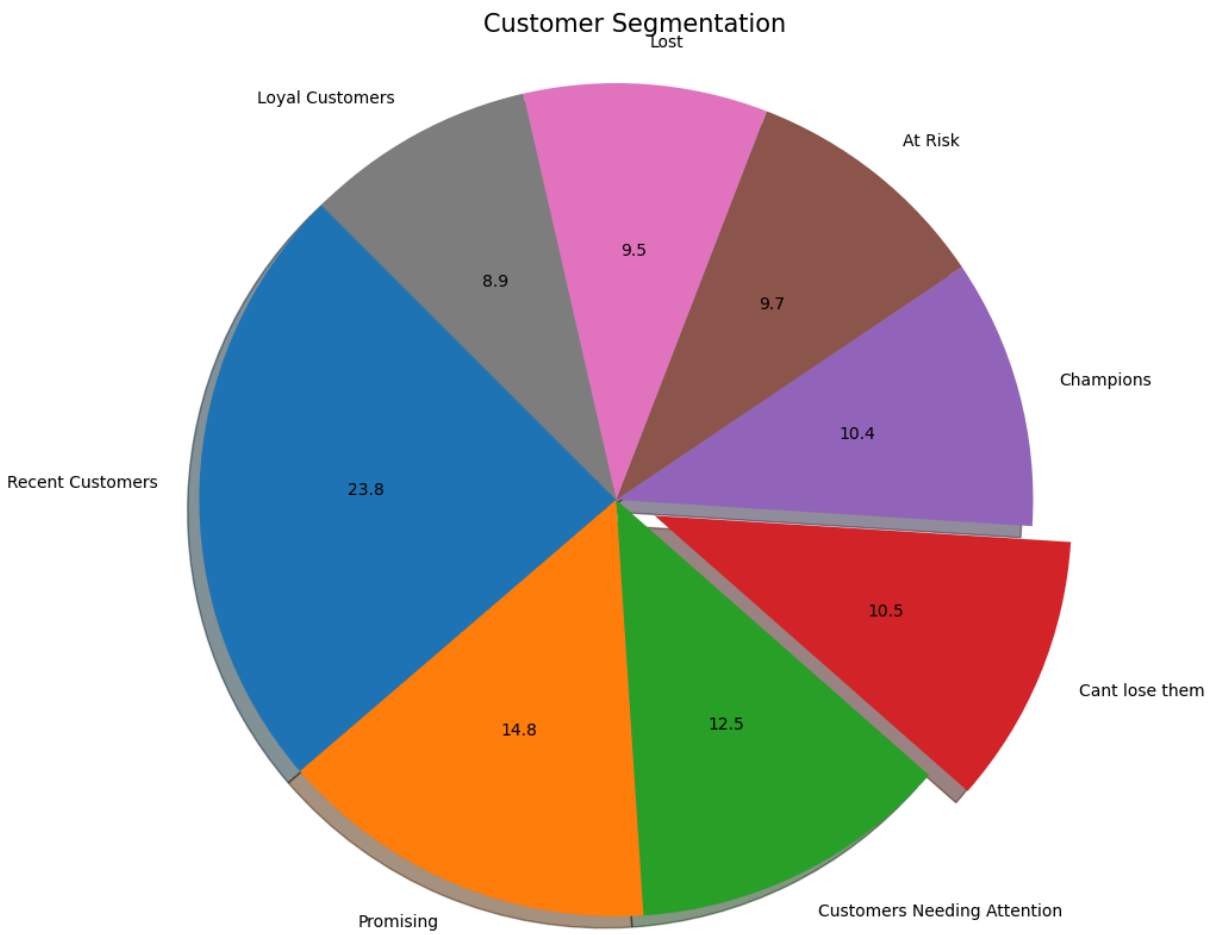
```
    elif (df['RFM_Total_Score'] == 10):# For RFM score with value 10
        return 'Loyal Customers'
    elif (df['RFM_Total_Score'] == 9): # For RFM score with value 9
        return 'Recent Customers'
    elif (df['RFM_Total_Score'] == 8): # For RFM score with value 8
         return 'Promising'
    elif (df['RFM_Total_Score'] == 7): # For RFM score with value 7
        return 'Customers Needing Attention'
    elif (df['RFM_Total_Score'] == 6): # For RFM score with value 6
        return 'Cant lose them'
    elif (df['RFM_Total_Score'] == 5): # For RFM score with value 5
        return 'At Risk'
    else:                              # For RFM score with value less than 5

        return 'Lost'
# Create a new variable RFM_Level
Customer_seg['Customer_Segmentation'] =Customer_seg.apply(RFM_Total_Score, axis=1)
# Print the header with top 5 rows to the console
Customer_seg.head()
```

Order Customer Id	R_Value	F_Value	M_Value	R_Score	F_Score	M_Score	RFM_Score	RFM_Total_Score	Customer_Segmentation
1	792	1	2362.250061	4	4	1	441	9	Recent Customers
2	280	1	839.800034	2	4	3	243	9	Recent Customers
3	613	4	328.339998	3	2	4	324	9	Recent Customers
4	380	2	1308.990032	2	3	2	232	7	Customers Needing Attention
5	457	4	1922.700043	3	2	2	322	7	Customers Needing Attention

How many customers are present in each segment?

```
# Calculate average values for each RFM_Level, and return a size of each segment
Customer_seg['Customer_Segmentation'].value_counts().plot.pie(figsize=(10,10),
                                    startangle=135,
explode=(0,0,0,0.1,0,0,0,0),autopct='%.1f',shadow=True)
plt.title("Customer Segmentation",size=15)
plt.ylabel(" ")
plt.axis('equal')
plt.show()
```



Since total customers are divided into 9 segments it can be seen that, 11.4% customers are at risk of losing them as customers and 11% customers needs attention else even they will be lost eventually.It can be seen that 4.4% of customers are already lost.

Our Top 10 Churned best customers who has not purchased anything in a while

```
churned=Customer_seg[(Customer_seg['RFM_Score']=='411')].sort_values('M_Value', ascending=False).head(10)
churned
```

Order Customer Id	R_Value	F_Value	M_Value	R_Score	F_Score	M_Score	RFM_Score	RFM_Total_Score	Customer_Segmentation
7270	767	10	9620.950013	4	1	1	411	6	Cant lose them
1902	891	6	7064.529968	4	1	1	411	6	Cant lose them
5877	727	7	7013.190094	4	1	1	411	6	Cant lose them
11124	684	14	6874.650034	4	1	1	411	6	Cant lose them
6669	857	8	6720.550102	4	1	1	411	6	Cant lose them
5219	880	9	6482.230087	4	1	1	411	6	Cant lose them
7457	696	8	6307.710022	4	1	1	411	6	Cant lose them
8026	705	9	6277.859970	4	1	1	411	6	Cant lose them
6948	700	6	6208.040077	4	1	1	411	6	Cant lose them
6876	697	6	6027.810028	4	1	1	411	6	Cant lose them

These customers used to place orders with huge amounts very frequently but they did not place orders from almost a year which means they are purchasing from other companies. These groups of people should be targeted with offers to gain them back.

Top 10 new best customers who place costly orders often.

```
#The R_Score should be low and F_Score, M_Score should be as high as possible
```

```
Customer_seg[(Customer_seg['RFM_Score']=='144')|(Customer_seg['RFM_Score']=='143')].sort_values('M_Value', ascending=False).head(10)
```

Order Customer Id	R_Value	F_Value	M_Value	R_Score	F_Score	M_Score	RFM_Score	RFM_Total_Score	Customer_Segmentation
3571	172	1	959.749984	1	4	3	143	8	Promising
7068	155	1	937.500000	1	4	3	143	8	Promising
10540	123	1	931.039978	1	4	3	143	8	Promising
4260	164	1	931.039978	1	4	3	143	8	Promising
11073	154	1	931.039978	1	4	3	143	8	Promising
11002	160	1	921.440002	1	4	3	143	8	Promising
3075	168	1	892.640015	1	4	3	143	8	Promising
5059	159	1	890.910004	1	4	3	143	8	Promising
796	135	1	879.800034	1	4	3	143	8	Promising
1141	145	1	849.800034	1	4	3	143	8	Promising

The above customers has the potential to become best customers this people should be targeted to convert them into loyal customers.All these different segment of customers should be targeted with different tailored advertisments and rewards for increased profits and more responsiveness from customers.

4.2.6 Data modelling & preprocessing

To measure the performance of different models the machine learning models are trained to detect fraud,late delivery for classification type. And sales, order quantity is predicted for regression type models.

A new dataset is created with the copy of original data for training the data and validation.

```
train_data=data.copy()
```

Two new columns are created for orders with suspected fraud and late delivery making them into binary classification, which in turn helps to measure performance of different models better.

```
train_data['fraud'] = np.where(train_data['Order Status'] == 'SUSPECTED_FRAUD', 1, 0)
train_data['late_delivery']=np.where(train_data['Delivery Status'] == 'Late delivery', 1, 0)
```

Now to measure machine models accurately all the columns with repeated values are dropped like late_delivery_risk column because, it is known all the products with late delivery risk are delivered late. And Order Status column because, a new column for fraud detection is created there is a chance machine learning model might take values directly from these columns to predict output.

```
#Dropping columns with repeated values
train_data.drop(['Delivery Status','Late_delivery_risk','Order Status','order_month_year','order date (DateOrders)'], axis=1, inplace=True)
```

It is important to check the type of variables in the data because machine learning models can only be trained with numerical values.

Type	object
Days for shipping (real)	int64
Days for shipment (scheduled)	int64
Benefit per order	float64
Sales per customer	float64
Category Id	int64
Category Name	object
Customer City	object
Customer Country	object
Customer Id	int64
Customer Segment	object
Customer State	object
Customer Zipcode	int64
Department Id	int64
Department Name	object
Market	object
Order City	object
Order Country	object
Order Customer Id	int64
Order Id	int64
Order Item Cardprod Id	int64
Order Item Discount	float64
Order Item Discount Rate	float64
Order Item Id	int64
Order Item Product Price	float64
Order Item Profit Ratio	float64
Order Item Quantity	float64
Sales	float64
Order Item Total	float64
Order Profit Per Order	float64
Order Region	object

Order State	object
Product Card Id	float64
Product Category Id	float64
Product Name	object
Product Price	float64
Shipping Mode	object
Customer Full Name	object
order_year	int32
order_month	int32
order_week_day	object
order_hour	int32
TotalPrice	float64
fraud	int64
late_delivery	int64

There are some columns with object type data which cannot be trained in machine learning models so all the object type data is converted to int type using preprocessing label encoder library.

```
# create the Labelencoder object
le = preprocessing.LabelEncoder()
#convert the categorical columns into numeric
train_data['Customer Country']  = le.fit_transform(train_data['Customer Country'])
train_data['Market']            = le.fit_transform(train_data['Market'])
train_data['Type']              = le.fit_transform(train_data['Type'])
train_data['Product Name']      = le.fit_transform(train_data['Product Name'])
train_data['Customer Segment']  = le.fit_transform(train_data['Customer Segment'])
train_data['Customer State']    = le.fit_transform(train_data['Customer State'])
train_data['Order Region']      = le.fit_transform(train_data['Order Region'])
```

```
train_data['Order City']        = le.fit_transform(train_data['Order City'])
train_data['Category Name']     = le.fit_transform(train_data['Category Name'])
train_data['Customer City']     = le.fit_transform(train_data['Customer City'])
train_data['Department Name']   = le.fit_transform(train_data['Department Name'])
train_data['Order State']       = le.fit_transform(train_data['Order State'])
train_data['Shipping Mode']     = le.fit_transform(train_data['Shipping Mode'])
train_data['order_week_day']    = le.fit_transform(train_data['order_week_day'])
train_data['Order Country']     = le.fit_transform(train_data['Order Country'])
train_data['Customer Full Name']= le.fit_transform(train_data['Customer Full Name'])

#display the initial records
train_data.head()
```

	Type	Days for shipping (real)	Days for shipment (scheduled)	Benefit per order	Sales per customer	Category Id	Category Name	Customer City	Customer Country	Customer Id	...	Product Price	Shipping Mode	Customer Full Name	order_year	order_month
0	1	3	4	91.250000	314.640015	73	34	65	1	20755	...	327.75	3	691	2018	1
1	3	5	4	-249.089996	311.359985	73	34	65	1	19492	...	327.75	3	1963	2018	1
2	0	4	4	-247.779999	309.720001	73	34	451	0	19491	...	327.75	3	1693	2018	1
3	1	3	4	22.860001	304.809998	73	34	284	0	19490	...	327.75	3	5270	2018	1
4	2	2	4	134.210007	298.250000	73	34	65	1	19489	...	327.75	3	4420	2018	1

Now all the data is transformed into int type. The dataset is split into train data and test data so model can be trained with train data and the performance of model can be evaluated using test data.

5.0 Simulation Results

The following code shows preparation in order to represent the results of our model

```
#All columns expect fraud
xf=train_data.loc[:, train_data.columns != 'fraud']
```

```
#Only fraud column
yf=train_data['fraud']
#Splitting the data into two parts in which 80% data will be used for training the model and 20% for testing
xf_train, xf_test,yf_train,yf_test = train_test_split(xf,yf,test_size = 0.2,random_state = 42)
#All columns expect fraud
xl=train_data.loc[:, train_data.columns != 'late_delivery']
#Only fraud column
yl=train_data['late_delivery']
#Splitting the data into two parts in which 80% data will be used for training the model and 20% for testing
xl_train, xl_test,yl_train,yl_test = train_test_split(xl,yl,test_size = 0.2, random_state = 42)
```

Since there are so many different variables with different ranges standard scaler is used to standardize total the data so it is internally consistent before training the data with machine learning.

```
sc = StandardScaler()
xf_train=sc.fit_transform(xf_train)
xf_test=sc.transform(xf_test)
xl_train=sc.fit_transform(xl_train)
xl_test=sc.transform(xl_test)
```

The data is now ready to be used in machine learning models since many different models are compared training every model from begining is complicated so a function is

defined to make the process bit easy. The output is in binary classification format so all the models are measured with Accuracy score,recall score and F1 score metrics.

To measure the performance of different models F1 score is used as the main metric because it is the harmonic mean of precison score and recall score.And all the scores are multiplied with 100 for better understanding

```
def classifiermodel(model_f,model_l,xf_train, xf_test,yf_train,yf_test,xl_train,
xl_test,yl_train,yl_test):
    model_f=model_f.fit(xf_train,yf_train) # Fitting train data for fraud detection
    model_l=model_l.fit(xl_train,yl_train) # Fitting train data for predection of late
delivery
    yf_pred=model_f.predict(xf_test)
    yl_pred=model_l.predict(xl_test)
    accuracy_f=accuracy_score(yf_pred, yf_test) #Accuracy for fraud detection
    accuracy_l=accuracy_score(yl_pred, yl_test) #Accuracy for predection of late
delivery
    recall_f=recall_score(yf_pred, yf_test) #Recall score for  fraud detection
    recall_l=recall_score(yl_pred, yl_test)# Recall score for predection of late delivery
    conf_f=confusion_matrix(yf_test, yf_pred)# fraud detection
    conf_l=confusion_matrix(yl_test, yl_pred)#predection of late delivery
    f1_f=f1_score(yf_test, yf_pred)#fraud detection
    f1_l=f1_score(yl_test, yl_pred)#predection of late delivery
    print('Model paramters used are :',model_f)
    print('Accuracy of fraud status is        :', (accuracy_f)*100,'%')
    print('Recall score of fraud status is        :', (recall_f)*100,'%')
    print('Conf Matrix of fraud status is        :\n',  (conf_f))
```

```
print('F1 score of fraud status is        :', (f1_f)*100,'%')
print('Accuracy of late delivery status is:', (accuracy_l)*100,'%')
print('Recall score of late delivery status is:', (recall_l)*100,'%')
print('Conf Matrix of late delivery status is: \n',(conf_l))
print('F1 score of late delivery status is:', (f1_l)*100,'%')
```

5.1 Random forest classification

Shown below is the code used to show the accuracy, f1 score, recall score and a configuration matrix.

```
model_f = RandomForestClassifier()
model_l = RandomForestClassifier(n_estimators=100, max_depth=10,
random_state=0)
classifiermodel(model_f,model_l,xf_train, xf_test,yf_train,yf_test,xl_train,
xl_test,yl_train,yl_test)
```

```
Model paramters used are : RandomForestClassifier()
Accuracy of fraud status is        : 98.63505747126436 %
Recall score of fraud status is        : 100.0 %
Conf Matrix of fraud status is        :
 [[5485    0]
 [  76    7]]
F1 score of fraud status is        : 15.555555555555555 %
Accuracy of late delivery status is: 98.54525862068965 %
Recall score of late delivery status is: 97.50154225786551 %
Conf Matrix of late delivery status is:
 [[2326   81]
```

```
[   0 3161]]
F1 score of late delivery status is: 98.73496798375761 %
```

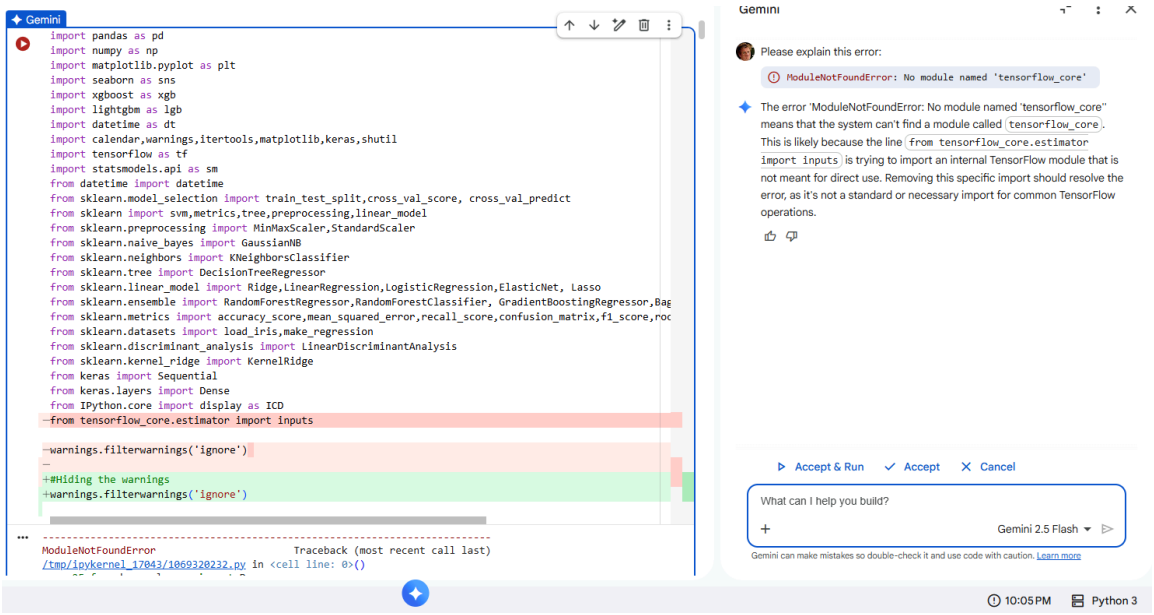
If this data were to be arranged in an actual table :

	fraud		Late delivery	
F1 score	15.56%		98.73%	
Accuracy score	98.64%		98.55%	
Recall score	100.0 %		97.50%	
Configuration matrix	5485	0	2326	81
	76	7	0	3161

5.2 Errors

All errors that were automatically detected by the compiler have been resolved by googles gemini, this is due to lack of python knowledge and was required to get this

project working



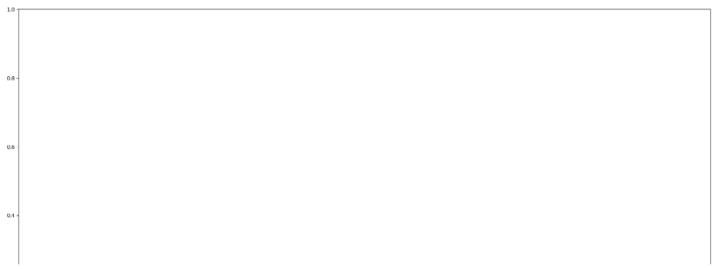
Gemini

```
 fig, ax = plt.subplots(figsize=(24,12))          # figsize
-sns.heatmap(data.corr(),annot=True,linewidths=.5,fmt='.1g',cmap= 'coolwarm') # Heatmap for correlation m
+sns.heatmap(data.select_dtypes(include=np.number).corr(),annot=True,linewidths=.5,fmt='.1g',cmap= 'cool
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
/tmp/ipykernel_17043/3024518477.py in <cell line: 0>()
      1 fig, ax = plt.subplots(figsize=(24,12))          # figsize
----> 2 sns.heatmap(data.corr(),annot=True,linewidths=.5,fmt='.1g',cmap= 'coolwarm') # Heatmap for
correlation matrix

                              3 frames
/usr/local/lib/python3.12/dist-packages/pandas/core/internals/managers.py in _interleave(self, dtype,
na_value)
   1751            else:
   1752                arr = blk.get_values(dtype)
-> 1753            result[rl.indexer] = arr
   1754            itemmask[rl.indexer] = 1
   1755

ValueError: could not convert string to float: 'DEBIT'
```



Gemini

Please explain this error:

`ValueError: could not convert string to float: 'DEBIT'`

The error `ValueError: could not convert string to float: 'DEBIT'` indicates that you're trying to calculate correlations with non-numeric data. The `data.corr()` function implicitly tries to convert all columns to numeric types, but the 'Type' column (and likely others) contains strings that cannot be converted. To fix this, you should calculate the correlation only on the numeric columns of your DataFrame. I will modify the cell to only include numeric data for the correlation heatmap.

Accept & Run Accept Cancel

What can I help you build?

Gemini 2.5 Flash

Gemini can make mistakes so double-check it and use code with caution. Learn more

Gemini

```
-
+data['order date (DateOrders)'] = pd.to_datetime(data['order date (DateOrders)'], errors='coerce')
+data.dropna(subset=['order date (DateOrders)'], inplace=True)
 data['order_year']= pd.DatetimeIndex(data['order date (DateOrders)']).year
 data['order_month'] = pd.DatetimeIndex(data['order date (DateOrders)']).month
 data['order_week_day'] = pd.DatetimeIndex(data['order date (DateOrders)']).weekday_name
 data['order_hour'] = pd.DatetimeIndex(data['order date (DateOrders)']).hour
-data['order_month_year'] = pd.to_datetime(data['order date (DateOrders)']).dt.to_period('M')
+data['order_month_year'] = data['order date (DateOrders)'].dt.to_period('M')
```

```
---------------------------------------------------------------------------
OverflowError                             Traceback (most recent call last)
conversion.pyx in pandas._libs.tslibs.conversion._TSObject.ensure_reso()

np_datetime.pyx in pandas._libs.tslibs.np_datetime.convert_reso()

OverflowError: result would overflow

The above exception was the direct cause of the following exception:

OutOfBoundsDatetime                       Traceback (most recent call last)
/tmp/ipykernel_17043/1562624585.py in <cell line: 0>()
----> 1 data['order_year']= pd.DatetimeIndex(data['order date (DateOrders)']).year
      2 data['order_month'] = pd.DatetimeIndex(data['order date (DateOrders)']).month
      3 data['order_week_day'] = pd.DatetimeIndex(data['order date (DateOrders)']).weekday_name
      4 data['order_hour'] = pd.DatetimeIndex(data['order date (DateOrders)']).hour
      5 data['order_month_year'] = pd.to_datetime(data['order date (DateOrders)']).dt.to_period('M')

                              3 frames
/usr/local/lib/python3.12/dist-packages/pandas/core/arrays/datetimes.py in objects_to_datetime64(data,
dayfirst, yearfirst, utc, errors, allow_object, out_unit)
   2396     data = np.asarray(data, dtype=np.object_)
   2397
-> 2398     result, tz_parsed = tslib.array_to_datetime(
   2399         data,
   2400         errors=errors,

tslib.pyx in pandas._libs.tslib.array_to_datetime()
```

Gemini

Please explain this error:

`OutOfBoundsDatetime: Out of bounds nanosecond timestamp: 8/,`

The `OutOfBoundsDatetime` error occurs because pandas encountered a date string it couldn't parse, specifically '8/' at position 8001 in the 'order date (DateOrders)' column. To fix this, I'll modify the code to use `pd.to_datetime` with `errors='coerce'`, which will turn invalid dates into `NaT` values, and then remove those rows to ensure all date operations are valid.

Accept & Run Accept Cancel

What can I help you build?

Gemini 2.5 Flash

Gemini can make mistakes so double-check it and use code with caution. Learn more

Gemini

```
#Filtering late delivery orders with standard class shipping
xyz1 = data[(data['Delivery Status'] == 'Late delivery') & (data['Shipping Mode'] == 'Standard Class')]
#Filtering late delivery orders with first class shipping
xyz2 = data[(data['Delivery Status'] == 'Late delivery') & (data['Shipping Mode'] == 'First Class')]
#Filtering late delivery orders with second class shipping
xyz3 = data[(data['Delivery Status'] == 'Late delivery') & (data['Shipping Mode'] == 'Second Class')]
#Filtering late delivery orders with same day shipping
xyz4 = data[(data['Delivery Status'] == 'Late delivery') & (data['Shipping Mode'] == 'Same Day')]
#Counting total values
-count1=xyz1['Order Region'].value_counts()
-count2=xyz2['Order Region'].value_counts()
-count3=xyz3['Order Region'].value_counts()
-count4=xyz4['Order Region'].value_counts()
+
+# Get all unique order regions to ensure consistent indexing
+all_regions = data['Order Region'].value_counts().index
+
+count1 = xyz1['Order Region'].value_counts().reindex(all_regions, fill_value=0)
+count2 = xyz2['Order Region'].value_counts().reindex(all_regions, fill_value=0)
+count3 = xyz3['Order Region'].value_counts().reindex(all_regions, fill_value=0)
+count4 = xyz4['Order Region'].value_counts().reindex(all_regions, fill_value=0)
+
#Index names
-names=data['Order Region'].value_counts().keys()
-n_groups=23
+names=all_regions
+n_groups=len(all_regions)
fig,ax = plt.subplots(figsize=(20,8))
index=np.arange(n_groups)
bar_width=0.2
opacity=0.6
type1=plt.bar(index,count1,bar_width,alpha=opacity,color='b',label='Standard Class')
type2=plt.bar(index+bar_width,count2,bar_width,alpha=opacity,color='r',label='First class')
type3=plt.bar(index+bar_width+bar_width,count3,bar_width,alpha=opacity,color='g',label='second class')
type4=plt.bar(index+bar_width+bar_width+bar_width,count4,bar_width,alpha=opacity,color='y',label='same c
plt.xlabel('Order Regions')
plt.ylabel('Number of shipments')
```

Gemini

Please explain this error:

`ValueError: shape mismatch: objects cannot be broadcast to a`

The error occurs because one of the `count` series, specifically for 'First Class' shipping, has fewer regions than the total number of unique regions, causing a mismatch in array shapes when plotting. I will fix this by reindexing all count series to include all regions and fill missing values with zero, ensuring consistent lengths for plotting.

Accept & Run Accept Cancel

What can I help you build?

Gemini 2.5 Flash

Gemini can make mistakes so double-check it and use code with caution. Learn more